

WHAT'S NEW IN

SPRING BOOT 4

Spring Framework 7 & Spring Boot 4

Dan Vega // Spring Developer Advocate @Broadcom

DAN VEGA

Spring Developer Advocate @Broadcom

Java Champion

Author: Fundamentals of Software Engineering
(O'Reilly)

24+ years building software

Cleveland, OH // Husband & Father



danvega.dev

[YouTube](#) // [Blog](#) // [Podcast](#)



OFFICE HOURS



<https://www.springofficehours.io>

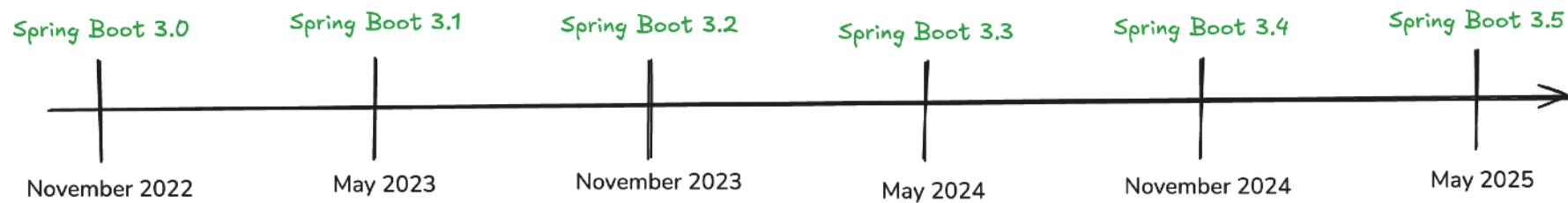
WHERE WE'VE BEEN

Spring Boot 3.x recap



*You can't really know
where you are going
until you know where
you have been.*

- Maya Angelou



3.0

November 2022

- JDK 17+
- Jakarta EE 9/10
- Ahead-of-Time (AOT) / GraalVM
- Observability
- HTTP Interface Clients
- RFC 7807 Problem Details

3.1

May 2023

- Docker Compose support
- Testcontainers
- Spring Authorization Server

3.2

November 2023

- JDK 21 (LTS) support
- Virtual Threads
- New Rest Client
- New JDBC Client
- SSL Bundle Reloading

3.3

May 2024

- CDS Support
- Observability Enhancements
- SBOM Actuator Endpoint
- Service Connections
- Base64 Resources

3.4

November 2024

- Structured Logging
- @Fallback Beans
- AssertJ support for MockMvc
- Expanded Virtual Thread Support
- ARM image support out-of-the-box

3.5

May 2025

- SSL Bundle Metrics
- Load properties from env vars
- Trigger Quartz jobs from Actuator
- ECS structured logging (nested format)

SPRING FRAMEWORK 7

&

SPRING BOOT 4

November 2025

THE ROAD TO GA

spring.io/blog/2025/09/02/road_to_ga_introduction

The Road to GA - Introduction

ENGINEERING | MICHAEL MINELLA | SEPTEMBER 02, 2025 | 3 COMMENTS

As you all should be aware by now, the Spring portfolio is in the process of driving towards the [next major versions to be released in November](#) of this year. This will be only the fourth major generation for Spring Boot and the seventh major generation for Spring Framework in its over 20 year history. We are excited to bring you a great set of new features that builds on the current foundations.

While we are keeping the JDK baseline at JDK 17 because that is the industry consensus, we will upgrade to Jakarta EE 11 (Tomcat 11, Hibernate ORM 7, Hibernate Validator 9) and embrace other upgrades like Kotlin 2, Jackson 3, and JUnit 6. Along with that, this round of majors contains a long list of interesting and exciting capabilities across the portfolio that you all have been asking for.

With that in mind, we want to spend some time making sure you are aware of these new capabilities. To do that, each week from now until Spring Boot 4.0 goes GA in November, we will have a weekly blog post about a new capability in the upcoming release. Some of the features we haven't really talked about much yet. For others, we already have blog posts introducing the topic and will revisit it to bring you the latest and greatest of where that capability will land in November. Below is the list of topics and when each blog post will be published. We will update this blog post with links to the individual blog posts as they are published. So be sure to bookmark this post and follow along!

Date	Title
September 9, 2025	Core Spring Resilience Features: <code>RConcurrencyLimit</code>, <code>RRetryable</code>, and <code>RetryTemplate</code>
September 16, 2025	API Versioning in Spring
September 23, 2025	HTTP Service Client Enhancements
September 30, 2025	The state of HTTP clients in Spring
October 7, 2025	Introducing Jackson 3 support in Spring
October 14, 2025	Share Consumer - aka Kafka Queues in Spring Kafka
October 21, 2025	Multi-Factor Authentication in Spring Security
October 28, 2025	Modularizing Spring Boot
November 5, 2025	Spring gRPC - Next Steps
November 12, 2025	Null-safe applications with Spring Boot 4
November 18, 2025	OpenTelemetry with Spring Boot
November 25, 2025	Ahead of Time Repositories (Part 2)

SPRING BOOT 4

github.com/danvega/sb4

danvega / sb4

sb4 Public

master 1 branch 0 Tags

Go to file Add file Code

danvega Readme title change 78b14ab · now 11 Commits

- features Moving Repo to readmes 11 hours ago
- .gitattributes initial commit 2 months ago
- .gitignore Moving Repo to readmes 11 hours ago
- README.md Readme title change now

README

What's new in Spring Boot 4

A discovery hub for exploring new features in Spring Boot 4 and Spring Framework 7. Each feature has its own dedicated repository with complete implementations, tutorials, and documentation.

Features

Feature	Description	Details
Null Safety	JSpecify annotations for compile-time null safety	Package-level <code>@NullMarked</code> annotation
HTTP Interface Clients	Declarative HTTP clients with <code>@HttpClient</code>	Replace <code>RestTemplate/WebClient</code> boilerplate
Bean Registration	Programmatic bean registration	<code>BeanRegistrar</code> interface
API Versioning	First-class API versioning support	Media type parameter versioning
JMS Client	JMS messaging with Apache Artemis	Producer/Consumer patterns
Resilience	Built-in resilience without external dependencies	<code>@Retryable</code> and <code>@ConcurrencyLimit</code>
		ISO-8601 defaults, <code>http11</code> method

About

No description, website, or topics provided.

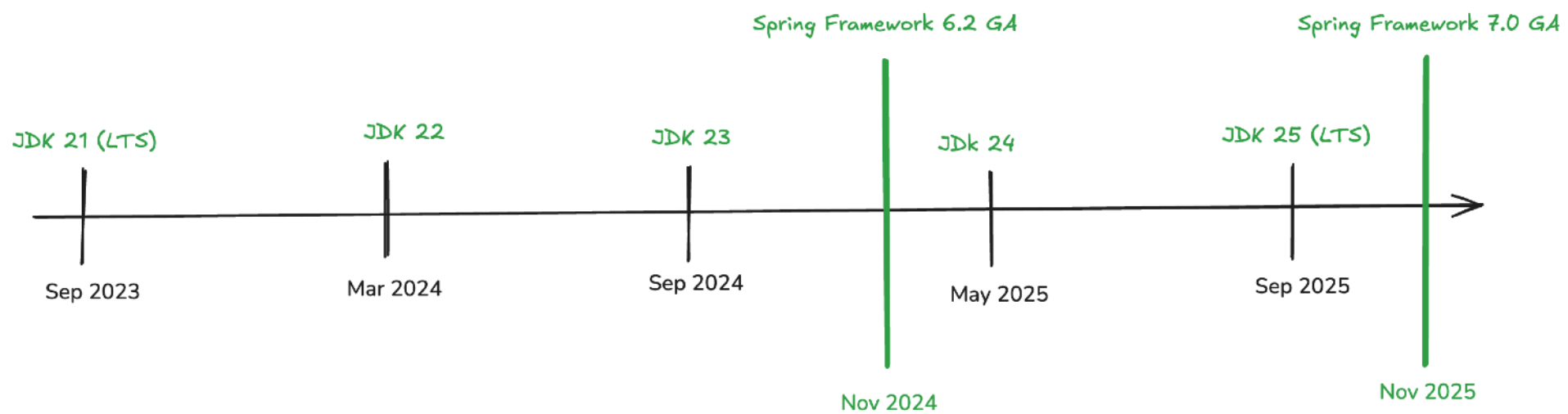
- Readme
- Activity
- 18 stars
- 3 watching
- 8 forks

Releases

No releases published
[Create a new release](#)

Packages

No packages published
[Publish your first package](#)



BASELINE UPGRADES

- JDK 17+ / JDK 25 (LTS)
- Jakarta EE 11
 - Servlet 6.1 (Tomcat 11)
 - JPA 3.2
 - Bean Validation 3.1
- Hibernate ORM 7.x
- Kotlin 2.2
- GraalVM 25
- Jackson 3
- JUnit 6

MODULAR AUTO-CONFIG

The biggest structural change in Boot 4

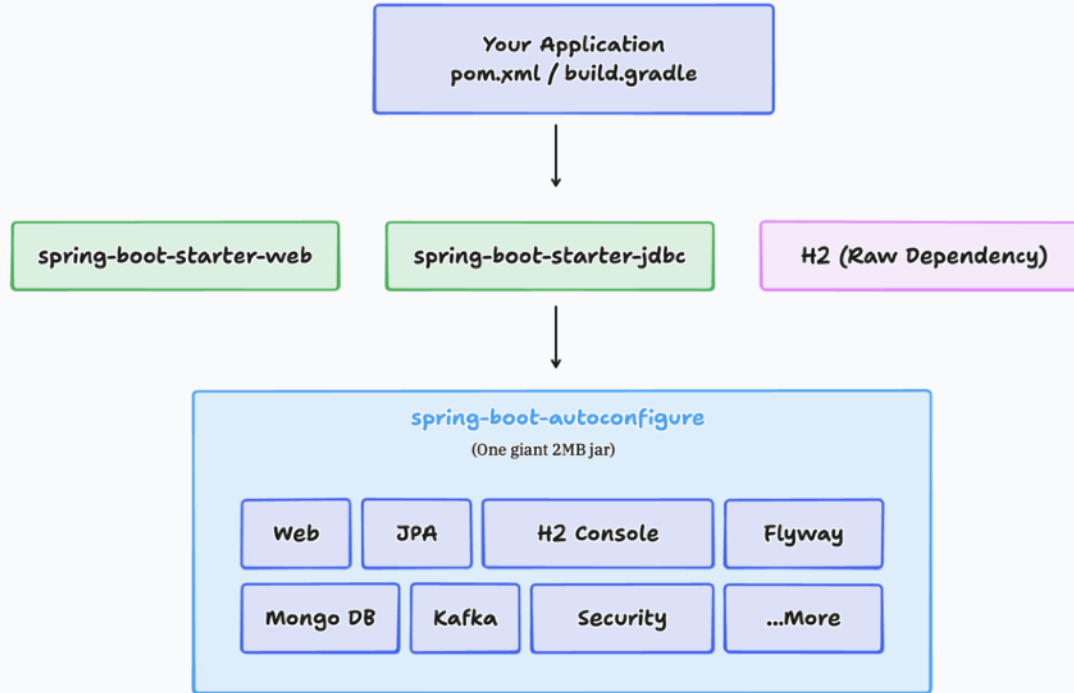
185 KB → 2 MB

spring-boot-autoconfigure JAR growth
from 2014 to 2025

WHAT CHANGED

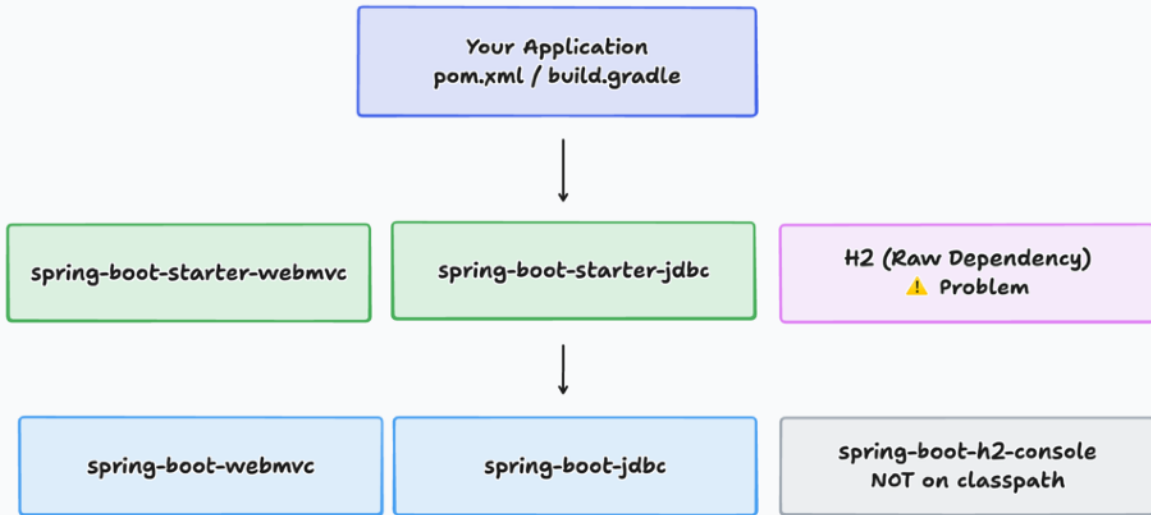
- Auto-config split into focused modules
- `spring-boot-starter-web` -> `spring-boot-webmvc`
- `spring-boot-starter-webmvc` (new, more explicit)
- Each starter brings only its own auto-config

Spring Boot 3.x Architecture



How it worked: All auto-configuration lived in one giant jar. With JDBC, Web, and H2 on your classpath, the /h2-console endpoint was automatically enabled—no extra dependencies needed.

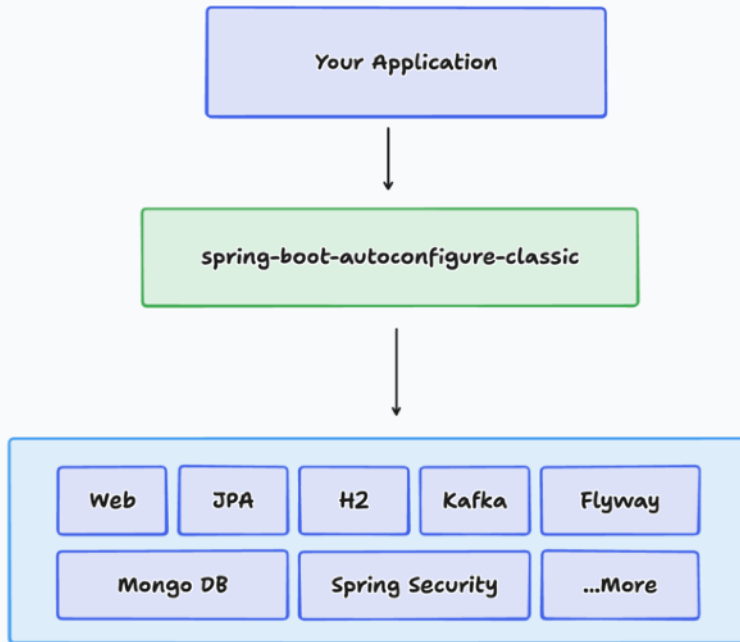
Spring Boot 4.x Architecture



Why H2 breaks: Auto-config code is now modular. You have the H2 dependency, but the auto-configuration code lives in `spring-boot-h2console` which isn't on your classpath!

The fix: Use `spring-boot-starter-h2console` which pulls in both the H2 library AND its auto-configuration module.

The Classic Escape Hatch



Quick Migration Path: Use `spring-boot-autoconfigure-classic` to get all auto-config modules at once (like the old giant jar behavior). Then migrate to specific starters over time.

JACKSON 3

New JSON library with big improvements

JACKSON 3 SUPPORT

- Immutable builder-based configuration
- ISO-8601 date defaults out of the box
- Unchecked exceptions - better for lambdas/streams
- New tools.jackson packages
- Spring Boot auto-configures JsonMapper bean
- Mix of Jackson 2 & 3 supported during migration

Jackson 3 - JsonMapper

```
@Component
public class DataLoader implements CommandLineRunner {

    private static final Logger log = LoggerFactory.getLogger(DataLoader.class);
    private static final String DONUTS_JSON_PATH = "classpath:/data/donuts-menu.json";

    private final JsonMapper jsonMapper;
    private final ResourceLoader resourceLoader;
    private List<Donut> donuts;

    public DataLoader(JsonMapper jsonMapper, ResourceLoader resourceLoader) {
        this.jsonMapper = jsonMapper;
        this.resourceLoader = resourceLoader;
    }

    @Override
    public void run(String... args) throws Exception {
        log.info("Loading Donuts \uD83C\uDF69");

        try {
            Resource resource = resourceLoader.getResource(DONUTS_JSON_PATH);
            this.donuts = jsonMapper.readValue(resource.getInputStream(), new TypeReference<>() {});
        } catch (JacksonException e) {
            log.error("Error loading Donuts: {}", e.getMessage());
        } catch (Exception e) {
            throw e;
        }
    }
}
```

Jackson - Use Jackson 2 Defaults

```
spring:  
  application:  
    name: donut-shop  
  
jackson:  
  serialization:  
    indent-output: true  
  use-jackson2-defaults: true
```

Jackson - JSON Views

```
public record Donut(  
    @JsonView(Views.Summary.class)  
    String type,  
    @JsonView(Views.Public.class)  
    Glaze glaze,  
    @JsonView(Views.Public.class)  
    List<String> toppings,  
    @JsonView(Views.Summary.class)  
    @JsonFormat(shape = JsonFormat.Shape.STRING, pattern = "$#.##")  
    BigDecimal price,  
    @JsonView(Views.Public.class)  
    Boolean isVegan,  
    @JsonView(Views.Internal.class)  
    Integer calories,  
    @JsonView(Views.Internal.class)  
    LocalDateTime bakedAt)  
{}
```

Jackson - JSON Views

```
public class Views {  
  
    /**  
     * Summary view: Minimal information for quick listings  
     * Includes: type, price  
     */  
    public interface Summary {}  
  
    /**  
     * Public view: Information suitable for public API consumers  
     * Includes: Summary + glaze, toppings, isVegan  
     */  
    public interface Public extends Summary {}  
  
    /**  
     * Internal view: Additional details for internal use  
     * Includes: Public + calories, bakedAt  
     */  
    public interface Internal extends Public {}  
  
    /**  
     * Admin view: Complete information for administrative purposes  
     * Includes: All fields (no restrictions)  
     */  
    public interface Admin extends Internal {}  
}
```

Spring Boot 3 - Request with filtered fields

```
// ✗ OLD: Required mutable wrapper object
var user = new User("Marcel", "Martin", LocalDate.of(1971, 7, 12),
    "m@m.fr", "1234 rue Gambetta", 69002, "Lyon", "France");

var jacksonValue = new MappingJacksonValue(user); // Wrapper!
jacksonValue.setSerializationView(Summary.class); // Mutable!

var response = this.restTemplate.postForObject(
    "http://localhost:8080/create",
    jacksonValue, // Have to send wrapper, not the actual object
    String.class
);
```

Spring Boot 4 - Request with filtered fields

```
//  NEW: Clean, immutable, fluent API
var user = new User("Marcel", "Martin", LocalDate.of(1971, 7, 12),
    "m@m.fr", "1234 rue Gambetta", 69002, "Lyon", "France");

var response = this.restClient.post()
    .uri("http://localhost:8080/create")
    .hint(JsonView.class.getName(), Summary.class) // Clean hint!
    .body(user) // Send the actual object
    .retrieve()
    .body(String.class);
```

NULL SAFETY WITH JSPECIFY

Say goodbye to NullPointerExceptions... maybe

EVER HIT A NullPointerException Exception?

Yeah. Everyone. Every single one of you.

THE PROBLEM ISN'T NULL

Sometimes you want to express the absence of a value

THE REAL PROBLEM WITH NULL

Is that it is usually implicit and undocumented

EXPRESS NULLABILITY JSPECIFY

<https://jspecify.dev>



JSPECIFY ANNOTATIONS

- `@Nullable`: Indicates that a type usage (Fields, Return Types, Parameters & Generics) can be null `@NotNullMarked` - everything is non-null by default
- `@NonNull`: This explicitly marks that null is not a valid value for this type usage
- `@NotNullMarked`: All type usages within that scope are considered non-null by default
- `@NullUnmarked`: All type usages have unspecified nullness (neither nullable nor non-null by default)

SPRING FRAMEWORK JSR-305 ANNOTATIONS

- Replaces Spring's JSR-305 annotations
 - @Nullable
 - @NonNull
 - @NonNullApi
 - @NonNullFields

Why are we doing this?

Ensure null safety in the IDE or during compilation time

SUPPORT FOR JSPECIFY

Across the Spring Portfolio of projects

BeanRegistrar API

Programmatic Bean Registration

PROGRAMMATICALLY REGISTER BEANS

A more flexible way to register multiple or conditional beans without abusing @Bean methods.

Bean Registrar in Action

```
public class MessageServiceRegistrar implements BeanRegistrar {
    @Override
    public void register(BeanRegistry registry, Environment env) {
        String messageType = env.getProperty("app.message-type", "email");

        switch (messageType.toLowerCase()) {
            case "email" → registry.registerBean("messageService",
                EmailMessageService.class,
                spec → spec.description("Email service via BeanRegistrar"));
            case "sms" → registry.registerBean("messageService",
                SmsMessageService.class,
                spec → spec.description("SMS service via BeanRegistrar"));
        }
    }
}
```

Bean Registrar in Action

```
@Configuration
@Import(MessageServiceRegistrar.class)
public class ModernConfig {

    // other bean definitions here

}
```

BUILT-IN API VERSIONING

First-class support in MVC & WebFlux

WE COULD ALWAYS VERSION OUR API'S

Introducing first-class support for web endpoint versioning in MVC /
WebFlux

API VERSIONING

- version attribute on `@GetMapping` / `@PostMapping`
- Version source: header, param, media type or path variable
- Configurable strategy via `ApiVersionConfigurer`
- Supported versions + default version
- No more custom interceptors or filters

API VERSIONING IN ACTION

```
@GetMapping(value =("/{version}/users", version = "1.0")
public List<UserDT0v1> findAllV1() {
    return userRepository.findAll().stream()
        .map(userMapper::toV1)
        .collect(Collectors.toList());
}
```

```
@GetMapping(value =("/{version}/users", version = "2.0")
public List<UserDT0v2> findAllV2() {
    return userRepository.findAll().stream()
        .map(userMapper::toV2)
        .collect(Collectors.toList());
}
```

API VERSIONING IN ACTION

```
@Configuration
public class WebConfig implements WebMvcConfigurer {

    /*
     * path-segment /v1/users
     * request header
     * query param
     * media type param
     */
    @Override
    public void configureApiVersioning(ApiVersionConfigurer configurer) {
        configurer
            .addSupportedVersions("1.0", "1.1", "2.0")
            .setDefaultVersion("1.0")
            .useRequestHeader("X-API-Version")
            .setVersionParser(new ApiVersionParser());
    }
}
```

HTTP INTERFACE CLIENTS

Declarative HTTP with `@HttpExchange`

HTTP INTERFACE CLIENTS

Introduced in Spring Framework 6 & Spring Boot 3.0

HTTP CLIENTS IN ACTION

```
@HttpExchange(url = "/todos", accept = "application/json")
public interface TodoService {
    @GetExchange("/")
    List<Todo> getAllTodos();

    @GetExchange("/{id}")
    Todo getTodoById(@PathVariable Long id);

    @PostExchange("/")
    Todo createTodo(@RequestBody Todo todo);
}
```

HTTP CLIENTS IN ACTION

```
@Bean
public TodoService todoService(RestClient.Builder restClientBuilder) {
    var restClient = restClientBuilder
        .baseUrl("https://jsonplaceholder.typicode.com")
        .build();

    var adapter = RestClientAdapter.create(restClient);
    var factory = HttpServiceProxyFactory.builderFor(adapter).build();
    return factory.createClient(TodoService.class);
}
```

HTTP CLIENTS IN ACTION

```
@HttpExchange(url = "/todos", accept = "application/json")
public interface TodoService {
    @GetExchange("/")
    List<Todo> getAllTodos();

    @GetExchange("/{id}")
    Todo getTodoById(@PathVariable Long id);
}

@Configuration(proxyBeanMethods = false)
@ImportHttpServices(TodoService.class)
public class HttpClientConfig {
    // That's it!
}
```

HTTP CLIENTS IN ACTION

```
@Configuration
@ImportHttpServices(group = "jsonplaceholder", types = {TodoService.class, PostService.class})
@ImportHttpServices(group = "github", types = {RepoService.class, IssueService.class})
public class MultiApiConfig {

    @Bean
    RestClientHttpServiceGroupConfigurer groupConfigurer() {
        return groups → {
            groups.filterByName("jsonplaceholder")
                .forEachClient((group, builder) → builder
                    .baseUrl("https://jsonplaceholder.typicode.com/")
                    .build());

            groups.filterByName("github")
                .forEachClient((group, builder) → builder
                    .baseUrl("https://api.github.com")
                    .defaultHeader("Accept", "application/vnd.github.v3+json")
                    .build());
        };
    }
}
```

BUILT-IN RESILIENCE

No more Spring Retry dependency

RESILIENCE FEATURES

- `@Retryable` - retry failed methods with backoff
- `RetryTemplate` - dynamic control over retry
- `@ConcurrencyLimit` - throttle concurrent calls
- Exponential backoff (1s, 2s, 4s, 8s...)
- Jitter support - prevent thundering herd
- All built into Spring Framework 7 Core

RESILIENCE IN ACTION

```
@Service
public class RestaurantService {
    @Retryable(
        maxAttempts = 4,
        includes = RestaurantApiException.class,
        delay = 1000,
        multiplier = 2
    )
    public List<MenuItem> getMenuFromPartner(String restaurantId) {
        if (random.nextDouble() < 0.4) {
            throw new RestaurantApiException("Partner restaurant API is temporarily unavailable");
        }
        Restaurant restaurant = dataLoader.getRestaurant(restaurantId);
        if (restaurant == null) {
            throw new RestaurantApiException("Restaurant not found: " + restaurantId);
        }
        List<MenuItem> menu = restaurant.menuItemIds().stream()
            .map(dataLoader::getMenuItem)
            .filter(item → item ≠ null && item.available())
            .toList();
        return menu;
    }
}
```

RESILIENCE IN ACTION

```
public DriverAssignmentService(DriverRetryListener driverRetryListener) {  
    this.driverRetryListener = driverRetryListener;  
  
    RetryPolicy retryPolicy = RetryPolicy.builder()  
        .maxAttempts(10)  
        .delay(Duration.ofMillis(2000))  
        .multiplier(1.5)  
        .maxDelay(Duration.ofMillis(10000))  
        .includes(NoDriversAvailableException.class)  
        .build();  
  
    retryTemplate = new RetryTemplate(retryPolicy);  
    retryTemplate.setRetryListener(driverRetryListener);  
}
```

RESILIENCE IN ACTION

```
@Service
public class RestaurantNotificationService {

    private static final Logger log =
        LoggerFactory.getLogger(RestaurantNotificationService.class);

    @ConcurrencyLimit(3)
    public void notifyRestaurant(Order order) {
        LocalDateTime start = LocalDateTime.now();
        log.info("[CONCURRENT] Sending notification to restaurant for order {} (Thread: {})",
            order.id(), Thread.currentThread().getName());

        // Simulate notification taking time (network call, webhook, etc.)
        simulateDelay(Duration.ofSeconds(2));

        LocalDateTime end = LocalDateTime.now();
        log.info("[CONCURRENT] Notification sent for order {} (took {}ms)",
            order.id(), Duration.between(start, end).toMillis());
    }
}
```

REST TEST CLIENT

A modern testing facade

HTTP CLIENTS

A quick history lesson on Http Clients

RestTestClient

Method	Speed	What It Tests	Best For
<code>bindToController</code>	⚡ Fastest	Just your controller	Quick unit tests
<code>bindToMockMvc</code>	🚀 Fast	Controller + Spring MVC	Testing validation, security
<code>bindToApplicationContext</code>	🐢 Slower	Full app (no HTTP)	Real database tests
<code>bindToServer</code>	🐌 Slowest	Everything with HTTP	Complete end-to-end tests
<code>bindToRouterFunction</code>	⚡ Fastest	Functional endpoints	WebFlux functional routes

REST TEST CLIENT IN ACTION

```
@WebMvcTest(TodoSimpleController.class)
@AutoConfigureRestTestClient
public class TodoSimpleControllerTest {

    @Autowired
    RestTestClient client;

    @Test
    public void findAllTodos() {
        List<Todo> todos = client.get()
            .uri("/api/todos/simple/")
            .exchange()
            .expectStatus().isOk()
            .expectBody(new ParameterizedTypeReference<List<Todo>>() {})
            .returnResult()
            .getResponseBody();

        assertEquals(1, todos.size());
        assertEquals("First Todo", todos.get(0).title());
    }
}
```

OPEN TELEMETRY

A new spring boot starter

KEY CONCEPTS

- Single Dependency: spring-boot-starter-opentelemetry - replaces complex setup
- Automatic instrumentation: HTTP server/client, JDBC & more
- Log correlation: Automatic trace/span ID injection into logs
- OLTP export: Works with any OpenTelemetry-compatible backend
- Production ready: Official Spring Support, no alpha dependencies

LGTM STACK

- Loki - for logs (Log aggregation system)
- Grafana - for visualization and dashboards
- Tempo - for traces (distributed tracing backend)
- Mimir - for metrics (long-term storage for Prometheus metrics)

OPEN TELEMETRY SPRING BOOT STARTER

```
spring:
  application:
    name: my-app

management:
  tracing:
    sampling:
      probability: 1.0 # 100% for development

otlp:
  metrics:
    export:
      url: http://localhost:4318/v1/metrics

opentelemetry:
  tracing:
    export:
      otlp:
        endpoint: http://localhost:4318/v1/traces

logging:
  export:
    otlp:
      endpoint: http://localhost:4318/v1/logs
```

Data source Tempo ▾

ⓘ Need help ▾

« ⓘ Last 30 minutes ▾

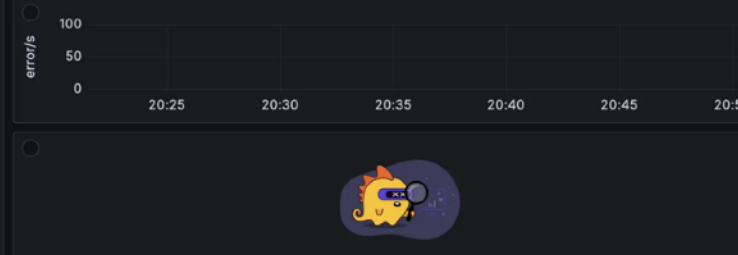
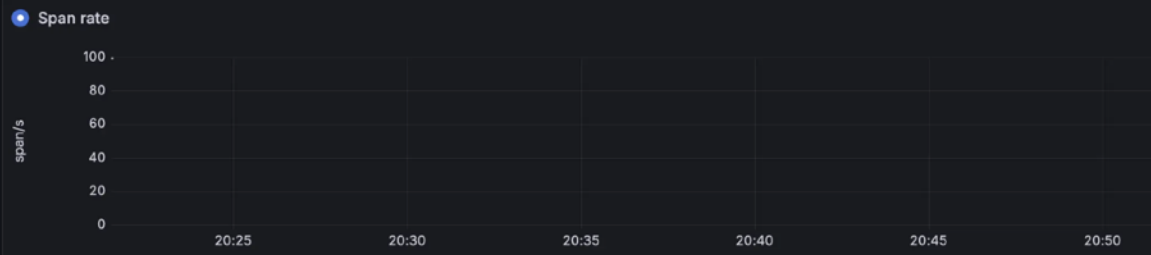
» 🔍

↺

Filters Root spans All spans ▾ 🔍 Filter by label values

Trace ID Enter an ID and press Enter

Select metric type ⓘ

Breakdown Service structure Comparison **Traces 3**

View a list of spans for the current set of filters.

Attributes

Selected (0): None

Q Search attributes...

Favorites All Resource Span

- ☐ resource.service.name ★
- ☐ span.name ★
- ☐ span.kind ★
- ☐ span.rootName ★
- ☐ span.rootServiceName ★
- ☐ span.status ★
- ☐ span.statusMessage ★

Start time	Trace Service	Trace Name	Duration	traceldHidden	Name
2025-12-23 20:51:22.869	ot	http get /slow	502 ms	2ed6eaabb5c63f35d176c3ebz	
2025-12-23 20:51:21.577	ot	http get /g http get /slow	60.0 ms	d79ccda3f596c9bb12cc804e1	
2025-12-23 20:51:20.145	ot	http get /	16.6 ms	1eaa080754c6abc64f5c345d	

JMS CLIENT

Virtual Threads

UNIFIED JMS CLIENT

- Modern alternative to JmsTemplate
- Fluent API like RestClient & JdbcClient
- Customizable QoS settings
- Unified exception translation
- Supports jakarta.jms and Spring messaging

JMS CLIENT IN ACTION

```
@Service
public class OrderMessagingService {

    private final JmsClient jmsClient;

    public OrderMessagingService(JmsClient jmsClient) {
        this.jmsClient = jmsClient;
    }

    public void sendOrder(Order order) {
        jmsClient.send("orders.queue")
            .withBody(order);
    }
}
```

SPRING DATA AOT REPOSITORIES

Generate Repository code at build time

SPRING DATA REPOSITORIES

```
@Repository
public interface CoffeeRepository extends ListCrudRepository<Coffee, Long> {

    List<Coffee> findByNameContainingIgnoreCase(String name);

    List<Coffee> findBySizeAndPriceGreaterThan(Size size, BigDecimal price);

}
```

THE SOLUTION?

AOT Repositories

50-70%

Faster startup with AOT repositories

Query parsing at build time, not runtime

Decompiled .class file, bytecode version: 69.0 (Java 25)

```
1  > /.../
5
6  package dev.danvega.coffee.coffee;
7
8  > import ...
21
22  @Generated
23  public class CoffeeRepositoryImpl__AotRepository extends AotRepositoryFragmentSupport {
24      public CoffeeRepositoryImpl__AotRepository(JdbcAggregateOperations var1, RowMapperFactory var2, RepositoryFactoryBeanSupport.FragmentCreationContext var3) {
25          super(var1, var2, var3);
26      }
27
28      public List<Coffee> findAffordableCoffeeBySize(String var1, BigDecimal var2) {
29          String var3 = "SELECT * FROM coffee\nWHERE size = :size\nAND price ≤ :maxPrice\nORDER BY price DESC\n";
30          MapSqlParameterSource var4 = new MapSqlParameterSource();
31
32          class ExpressionMarker {
33              ExpressionMarker() {
34                  Objects.requireNonNull( obj: CoffeeRepositoryImpl__AotRepository.this);
35                  super();
36              }
37          }
38
39          this.getBindableValue(ExpressionMarker.class.getEnclosingMethod(), var1, parameterIndex: 0).bind( parameterName: "size", var4);
40          this.getBindableValue(ExpressionMarker.class.getEnclosingMethod(), var2, parameterIndex: 1).bind( parameterName: "maxPrice", var4);
41          RowMapper var5 = this.getRowMapperFactory().create(Coffee.class);
42          List var6 = (List)this.getJdbcOperations().query(var3, var4, new RowMapperResultSetExtractor(var5));
43          return (List)this.convertMany(var6, Coffee.class);
44      }
```

SPRING SECURITY MFA

Multi-Factor Authentication

MULTI-FACTOR AUTH

- First-class MFA in Spring Security
- `@EnableMultiFactorAuthentication`
- Factor tracking via `FactorGrantedAuthority`
- Global or selective (per-endpoint) MFA
- PASSWORD + One-Time Token out of the box

SPRING DATA REPOSITORIES

```
@Repository
public interface CoffeeRepository extends ListCrudRepository<Coffee, Long> {

    List<Coffee> findByNameContainingIgnoreCase(String name);

    List<Coffee> findBySizeAndPriceGreaterThan(Size size, BigDecimal price);

}
```

What is Multi-Factor Authentication?

MFA requires users to provide **multiple factors** to authenticate — combining something you know, have, or are.

OWASP FACTOR CATEGORIES

Something you know — Password, PIN

Something you have — SMS, Email, Token

Something you are — Biometrics

Somewhere you are — Geolocation

Something you do — Behavior profiling

SPRING SECURITY APPROACH

At authentication time, Spring Security adds a `FactorGrantedAuthority` to track which factors have been verified.

Authorization rules can then require multiple factors before granting access.

```
FACTOR_PASSWORD + FACTOR_OTT
```

Enabling MFA

Use `@EnableMultiFactorAuthentication` to require multiple factors globally.

```
@Configuration
@EnableWebSecurity(debug = true)
@EnableMultiFactorAuthentication(authorities = {
    FactorGrantedAuthority.PASSWORD_AUTHORITY,
    FactorGrantedAuthority.OTT_AUTHORITY
})
class SecurityConfig {

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        return http
            .authorizeHttpRequests((authorize) → authorize
                .requestMatchers("/", "/ott/sent").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .formLogin(withDefaults())
            .oneTimeTokenLogin(withDefaults())
            .build();
    }
}
```

All URLs: Require both PASSWORD and OTT factors

Smart Redirect: Automatically sends user to missing factor's login

Selective MFA

Require MFA only for specific endpoints using AuthorizationManagerFactories.

```
@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {

    var mfa = AuthorizationManagerFactories.multiFactor()
        .requireFactors(
            FactorGrantedAuthority.PASSWORD_AUTHORITY,
            FactorGrantedAuthority.OTT_AUTHORITY
        )
        .build();

    http
        .authorizeHttpRequests((authorize) → authorize
            .requestMatchers("/admin/**").access(mfa.hasRole("ADMIN"))
            .requestMatchers("/user/settings/**").access(mfa.authenticated())
            .anyRequest().authenticated()
        )
        .formLogin(Customizer.withDefaults())
        .oneTimeTokenLogin(Customizer.withDefaults());

    return http.build();
}
```

/admin/** → Requires MFA + ADMIN role

/user/settings/** → Requires MFA only

Everything else → Single factor OK

start.spring.io

Spring Boot 4.0.3 is available NOW

Java 25

SHOW ME THE CODE

`github.com/danvega/sb4`

RESOURCES

Spring Framework 7.0 Release Notes

github.com/spring-projects/spring-framework/wiki

Spring Boot 4.0 Release Notes

github.com/spring-projects/spring-boot/wiki

Road to GA Blog Series

spring.io/blog/2025/09/02/road_to_ga_introduction

Demo Repository

github.com/danvega/sb4

Spring Portfolio Generations

spring.io/projects/generations

ROADMAP

Spring Framework 6.2

Available Now

- Final 6.x with long-term support
- Foundation for Boot 3.4 & 3.5
- JDK 17 & JDK 21 LTS
- Deep core container revision

Spring Framework 7.0

November 2025

- Foundation for Spring Boot 4.0
- JDK 17+, optimized for JDK 25
- Jakarta EE 11, JSpecify, Kotlin 2.x
- Bean registration, API versioning

THANK YOU!

dan.vega@broadcom.com

danvega.dev

[@therealdanvega](#)